EXPERIMENT-05 :

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets

CODE

```
# Implementation of K-Nearest Neighbours (K-NN) in Python


import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay


# 1. Load dataset

iris = load_iris()

X = iris.data

y = iris.target


print("Dataset: Iris")

print("Number of samples:", X.shape[0])

print("Features:", iris.feature_names)

print("Classes:", iris.target_names)


# 2. Split dataset into training and testing data

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Normalize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Create and train K-NN classifier
k = 5
model = KNeighborsClassifier(n_neighbors=k)
model.fit(X_train, y_train)

# 5. Predict test data
y_pred = model.predict(X_test)

# 6. Display results
print("\nActual Labels:")
print(y_test)

print("\nPredicted Labels:")
print(y_pred)

accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", round(accuracy * 100, 2), "%")
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# 7. Plot confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
)
disp.plot(cmap="Blues")
plt.title("K-NN Confusion Matrix")
plt.show()

# 8. Plot actual vs predicted classes
plt.figure(figsize=(8, 4))
samples = np.arange(len(y_test))

plt.plot(samples, y_test, "o-", label="Actual Class")
plt.plot(samples, y_pred, "x--", label="Predicted Class")

plt.xlabel("Test Sample")
plt.ylabel("Class")
plt.title("K-NN: Actual vs Predicted")
plt.yticks([0, 1, 2], iris.target_names)
plt.legend()
plt.grid(True)
```

```
plt.show()

# 9. Predict a new sample
new_sample = np.array([[5.1, 3.5, 1.4, 0.2]])
new_sample_scaled = scaler.transform(new_sample)

prediction = model.predict(new_sample_scaled)

print("\nNew Sample:", new_sample[0])
print("Predicted Flower:", iris.target_names[prediction[0]])
```

OUTPUT:

